# 35. Google Docs (Collaborative Document Editing)

> **Interviewer:** Design the backend for something like Google Docs — specifically the part that makes it magic: **multiple people editing the same document at the same time**, each seeing everyone else's keystrokes appear live, and everyone ending up looking at the exact same document even though they typed in different places at nearly the same instant.

**Candidate:** This is a genuinely different animal from most CRUD systems, because the hard requirement isn't throughput or storage — it's **convergence under concurrency**: many writers, no central lock, sub-second feedback, and a mathematical guarantee that everyone's screen ends up identical. Let me clarify scope before I design.

---

## 1. Clarifying the requirements

> **Candidate:** A few questions: 1. Is this rich text (bold, headings, images) or plain text? I'll assume rich text but focus the hard part on the text/structure model, since formatting layers on top. 2. How many concurrent editors per document — 2, or hundreds (like a company all-hands doc)? 3. Do we need to support **offline editing** — someone edits on a plane, reconnects later? 4. Do we need **version history** (restore to any point in time)? 5. Is eventual convergence enough, or do concurrent edits need to *feel* instant locally (i.e., can't wait for a round trip before showing your own keystroke)?
>
> **Interviewer:** Rich text, yes. Typically 2-20 concurrent editors, occasionally up to ~100 on a huge shared doc. Yes — must support offline edits that reconcile on reconnect. Yes, version history. And yes: your own keystrokes must appear **instantly**, before the server even sees them — no typing lag, ever.

**Candidate:** Good, that last point is the crux. Let me write requirements down.

**Functional** - Multiple users open the same document and edit concurrently. - Every user sees every other user's edits live (sub-second). - Local edits render **instantly** (optimistic, no round trip). - Presence: show who's in the doc and where their cursor/selection is. - Offline edits queue locally and reconcile automatically on reconnect. - Version history / point-in-time restore.

**Non-functional** - **Convergence:** after all operations are applied everywhere, every replica must reach the *identical* document, regardless of the order edits happened to arrive over the network. - **Low latency for local echo:** typing must never wait on the network (this rules out "send keystroke, wait for server ack, then render"). - **Consistency model is the crux, not throughput:** a single doc rarely sees more than a few hundred ops/sec even with 100 editors — this isn't a scale problem, it's a **conflict-resolution** problem. - **Durability:** no edit should be silently lost, even across disconnects/crashes. - **Availability:** one document's editors shouldn't be affected by load elsewhere.

---

## 2. Estimation — the number that tells you this isn't a throughput problem

> **Candidate:** Let me size it, mostly to show *why* the hard part isn't scale.

```
A fast typist does ~5–8 keystrokes/sec. A doc with 20 concurrent editors, all typing
at once (rare, but worst case): ~150 ops/sec on ONE document. Even the "100 editors on
one huge doc" case tops out at a few hundred ops/sec.

Compare: a flash sale or burger giveaway hits 100,000s of ops/sec on ONE hot key.
Google Docs' hot key (one document) sees maybe 200 ops/sec, ever.

=> The bottleneck is NOT raw throughput on a document. It's that we have MANY
   writers with NO lock, each wanting sub-100ms local feedback, and we need every
   replica to converge to the same bytes despite arbitrary interleavings of network
   delivery order. This is a correctness/algorithm problem, not a capacity problem.

Storage: a doc is typically KBs–MBs of text/structure. Millions of docs x MBs =
easily petabyte-scale in aggregate, but that's a boring object-storage problem.
The interesting number here is EDITS, not BYTES: a long-lived doc can accumulate
hundreds of thousands of small operations over its life -> that history needs to be
compacted (snapshots), or replaying it becomes the bottleneck.
```

**Candidate:** So the design bends entirely around **how a keystroke gets from your keyboard to everyone else's screen with sub-100ms local echo, and a provable way to make everyone's document converge** — not around raw QPS.

---

## 3. The naive V1 — and why it doesn't even get off the ground

> **Candidate:** Let me state the "obvious" approach so we're aligned on why it fails.

```
  [User A types 'x'] --> save WHOLE document --> [Server] --> broadcast whole doc
  [User B types 'y'] --> save WHOLE document --> [Server] --> broadcast whole doc

  Whoever's "save whole doc" lands LAST wins. The other person's edit is just gone.
```

**Why it dies immediately:** if both users' clients send "here is my full copy of the document" and the server just overwrites with whichever arrives last (**last-write-wins on the whole document**), one user's edit is silently discarded the instant two people type "at the same time" — which, in a shared doc, is constantly. This isn't a scale problem, it's a **correctness** problem from the first concurrent edit. I'll go deeper on exactly this in a deep-dive later, but it tells me immediately: **we must never transmit or diff whole documents; we must transmit and apply small, well-defined operations** ("insert 'x' at position 42"), and we need an algorithm that can reconcile operations that raced each other.

---

## 4. The key idea — operations over a WebSocket, not documents

> **Candidate:** The building block for all real-time collaborative editors: represent every edit as a tiny **operation** — `insert(pos, text)`, `delete(pos, len)`, `format(range, attr)` — not a snapshot. Send operations, not documents.

```
User types 'x' at position 42
     │
     ▼
Local editor:
  1. Applies the op to its OWN buffer immediately  (instant local echo, no wait)
  2. Sends the op over an open WebSocket to the doc's server session
     │
     ▼
[ WebSocket connection, one per client, held open for the editing session ]
     │
     ▼
[ Document server (the "sequencer") for THIS doc ]
  - assigns the op a place in the doc's total order
  - transforms/merges it against any concurrent ops it already sequenced
  - broadcasts the transformed op to every OTHER connected client
     │
     ▼
Other clients: receive op, apply it (transformed) to their local buffer
```

**Why WebSocket and not HTTP polling?** Editing is bidirectional and continuous — the server needs to push other users' ops to you the instant they happen, and you need to push yours the instant you type. A persistent WebSocket avoids poll latency and per-request HTTP overhead; polling every 200ms both feels laggy and wastes requests when nobody's typing, and can't push server-initiated events (someone else's op) without a hack (long polling).

**Why send operations and not the whole document?** Two reasons, and I'll expand in the first deep-dive: (1) bandwidth — a keystroke is a few bytes, a document is not; (2) correctness — an operation carries *intent* ("insert at 42"), which is what lets us mathematically transform two racing edits so both survive. A whole-document diff can't tell "user A inserted a word" from "user A deleted everything and retyped it with one word different."

---

## 5. The heart of it: OT vs CRDT — how concurrent ops actually converge

> **Interviewer:** OK — so ops travel over WebSocket to a server. But two users can still type at "the same time," so the server sees two ops that were each computed against a document state that didn't yet include the other's edit. How do you make sure they don't corrupt each other?

**Candidate:** This is the actual algorithmic core of the whole system. There are two proven families of answer.

### Operational Transformation (OT) — a central sequencer transforms operations

Every op is generated against a known **document version number**. When the server receives an op generated against version *v*, but the doc is already at version *v+2* (two other ops landed first), the server **transforms** the incoming op against those two ops before applying it — adjusting its position/length so it still does what the user intended, *given* what already happened.

```
Doc at version 5. User A's client (still showing v5) sends insert(pos=10,"X").
Meanwhile ops from B already advanced the server to v7 (two inserts before pos 10).
Server: transform(A's op, against B's two ops) -> insert(pos=12, "X")  [shifted by +2]
Server applies transformed op -> v8, broadcasts insert(pos=12,"X") to everyone.
```

This is the model **Google Docs itself actually uses** (via its `jupiter`/OT-based sync protocol): a central server is the arbiter of order, every client sends ops relative to a known version, and the server transforms and re-broadcasts. It requires a **central sequencer** (or a designated leader) that all ops flow through, because transformation order matters.

### CRDTs (Conflict-free Replicated Data Types) — convergence by construction, no sequencer

Instead of transforming operations against each other, a CRDT gives every unit of content (e.g. every character, or every list element) a **globally unique, stably-ordered ID** (commonly `(logical-clock, replicaId)`), and defines merge rules such that **applying the same set of operations in any order produces the same result** — mathematically, by construction (the merge function is commutative, associative, idempotent). No central coordinator is required to reach agreement; any two replicas that have seen the same set of ops converge, peer-to-peer or not.

```
Character 'X' inserted by replica A gets ID (clock=5, replica=A)
Character 'Y' inserted by replica B gets ID (clock=5, replica=B)
Every replica, no matter what order it received them in, sorts ties by replicaId
-> both replicas end up with the SAME final order: ...A's X before B's Y... (or
   vice versa, consistently) -- convergence guaranteed without asking a server
   "what happened first."
```

### The tradeoff

| | | |
|---|---|---|
| Coordinator | Needs a **central sequencer** (server or leader) to define a canonical order | No coordinator needed — any replica can merge with any other |
| Offline support | Harder — needs the version number you diverged from, and transforms can get complex over long offline periods | Natural fit — ops carry their own IDs, can merge whenever, however delayed |
| Metadata overhead | Small — just an op + version number | Larger — every character/element carries an ID (clock + replica), can bloat storage/bandwidth |
| Maturity / battle-tested | Yes — Google Docs, Google Wave lineage, decades of production use | Newer in this space; used by Figma, Notion (partially), Yjs-based apps |

| | | |
|---|---|---|
| Complexity of the algorithm | Transform functions get genuinely hairy for rich text (bold+insert+delete interleavings) | Simpler mental model per-op, but garbage collection of tombstones/IDs is its own problem |

**My pick for this design: OT with a central per-document sequencer**, because we already have a natural coordinator — a client opens a WebSocket to *a* server, and that server is a perfectly good place to be the single source of truth for "what order did edits happen in, for this one document." We're not peer-to-peer and we're not primarily optimizing for offline-first (though we still need to support it, which I'll cover). If this were a local-first / P2P / frequently-offline product (e.g. a note app that syncs opportunistically between devices with no always-on server), I'd lean CRDT instead — that's exactly the regime CRDTs were built for.

---

## 6. Presence — cursors and selections, a different (cheaper) stream

> **Interviewer:** I also see everyone's cursor moving live, with their name/color. Is that the same pipeline?

**Candidate:** Related, but I'd deliberately keep it a **separate, lower-guarantee stream**:

```
[ Same WebSocket connection ]
      │
      ├── "durable" channel: document operations (must never be lost, must be ordered,
      │    persisted, part of history)
      │
      └── "ephemeral" channel: presence (cursor position, selection range, "is typing")
           -- NOT persisted, NOT ordered against document ops, just latest-value-wins,
              broadcast to the room, dropped if a client disconnects
```

**Why separate it?** Presence is high-frequency (every mouse move / cursor tick) and **disposable** — if you miss 3 cursor updates because of a dropped packet, nobody cares, the 4th just repaints the cursor in the right place. Running it through the same ordered, persisted, must-never-lose-an-edit pipeline as real document operations would be wasteful and would slow down the thing that actually matters (the text). I model it as ephemeral pub/sub scoped to "everyone currently in this document's room" — ideal for something like Redis Pub/Sub or the WebSocket server's own in-memory fan-out, no durability needed.

---

## 7. Persistence, snapshots, and document storage

> **Interviewer:** Where does the actual document live, and how do you avoid replaying a million tiny operations every time someone opens a doc?

**Candidate:** Two tiers, the same pattern as event-sourced systems:

```
[ Op log ]  (append-only, per document, ORDERED)
  op#1, op#2, op#3, ... op#48213
     |
     |  every N ops (or every T seconds), a background job:
     ▼
[ Snapshot store ]
  snapshot@op#48000 = fully materialized document (rich text tree/JSON)
     |
Opening the doc = load latest snapshot (op#48000) + replay the small tail
(op#48001..48213) on top. Fast, regardless of how old the document is.
```

- **Op log** (source of truth for *what happened*): append-only, ordered per document. This doubles as **version history** — any prior point in time is "the snapshot before it, plus ops up to there." I'd store this in a system built for ordered append-heavy writes — something like Cassandra or a partitioned log (Kafka-backed) keyed by `docId`, so writes for different documents never contend and reads are a sequential range scan.
- **Snapshot store** (source of truth for *current state*, for fast loads): the fully materialized document (its rich-text tree, e.g. a JSON/protobuf blob), stored in an object store (S3-like) or a document DB (e.g. a Postgres row / DynamoDB item) keyed by `docId`. Rebuilt periodically by folding the op log forward.
- **Why not just replay the entire op log every time someone opens a 5-year-old doc?** Because that doc might have hundreds of thousands of ops; replaying all of them on every open is unbounded latency that grows forever. Snapshotting caps "time to open" at "load one snapshot + replay a small recent tail," regardless of document age.
- **Version history / restore** falls out of this for free: to view the doc "as of last Tuesday," load the nearest snapshot before that timestamp and replay ops up to it.

---

## 8. Offline editing and reconciliation

> **Interviewer:** Someone's on a plane, edits for two hours with no connection, then lands and reconnects. What happens?

**Candidate:** The client keeps editing **optimistically against its local buffer** the whole time — nothing about typing changes when offline, because local echo never depended on the network anyway. The difference is only in what happens at reconnect:

```
OFFLINE: client queues its ops locally (durable local queue, e.g. IndexedDB),
         each one tagged with "generated against version V" (the version the
         client last knew about before going offline).
     |
RECONNECT:
     |
     ▼
Client sends its queued ops (still tagged with their original base version V)
     |
     ▼
Server: "you're based on V, but the doc has moved to V+5,000 while you were away"
        -> transform (OT) each queued op forward against everything that happened
           since V, in order, just like the live concurrent case, just with a
           bigger backlog to transform against.
     |
     ▼
Server applies the transformed ops, broadcasts them to everyone currently connected,
sends the client the resulting canonical state (or the tail of ops to replay) so the
client's local buffer converges too.
```

The mechanism is identical to normal concurrent editing — OT doesn't care whether the "concurrent" op arrived 50ms late or 2 hours late, it just transforms against everything that happened since the base version. Long offline periods just mean a longer transform chain (or, practically, the server can compact: fold all the offline ops into one intent-preserving batch before transforming, and cap how far behind a client is allowed to be before it must reload a fresh snapshot rather than replay/transform a huge backlog).

---

## 9. Technology choices — what and why

| Concern | Choice | Why this, and why not the alternative |
|---|---|---|
| **Edit sync transport** | **WebSocket**, one connection per client per doc session | Needs bidirectional, low-latency, server-push. *Not HTTP polling* — polling adds visible lag and can't push others' edits without long-polling hacks; *not gRPC streaming* — WebSocket is the ubiquitous browser-native choice. |
| **Conflict resolution engine** | **OT with a central per-document sequencer** | We already have a natural coordinator (the doc's WebSocket server); OT is mature, battle-tested (this is what Google Docs runs), and keeps per-character metadata small. *Not CRDT* — CRDTs shine for offline-first/P2P with no server; here we have a server, and CRDT's per-character ID overhead isn't worth it for our access pattern. |
| **Sequencer placement** | **One logical owner server per document** (consistent hashing / sticky routing to a stateful server, or a leader elected per doc) | Transform order must be centrally decided for a given document; sharding by `docId` means different documents scale independently and no single server is a global bottleneck. |
| **Presence / cursors** | **Ephemeral pub/sub** (Redis Pub/Sub or in-memory fan-out), not persisted | High frequency, disposable data; keeping it off the durable pipeline protects the latency of real edits. |
| **Operation log (source of** | **Append-only log per document** (Kafka- | Ordered, append-heavy, per-doc write pattern; partitioning by `docId` isolates hot documents from each other. *Not a* |

| Concern | Choice | Why this, and why not the alternative |
|---|---|---|
| **truth for history)** | partitioned-by-docId, or Cassandra) | *single relational table with a global autoincrement* — that becomes a shared hot resource across all documents. |
| **Snapshot store** | **Object storage / document DB** (S3-style blob or DynamoDB item) keyed by `docId` | Materialized current state must load fast regardless of a doc's total edit history; periodic snapshotting bounds "time to open a doc." |
| **Version history** | **Snapshot + op-log replay to any point in time** | Free byproduct of the event-sourced design; no separate "history" system needed. |
| **Offline queue** | **Client-local durable queue** (IndexedDB) tagged with base version | Lets local editing continue uninterrupted; reconciliation reuses the same OT transform path as live concurrency. |

**The one-sentence justification you'd say out loud:** "*Every edit is a small operation sent over a persistent WebSocket to a per-document sequencer, which uses Operational Transformation to reconcile concurrent edits into one canonical order and rebroadcasts the transformed op to everyone; presence rides a separate, disposable channel so it never competes with real edits; and the document itself is event-sourced — an append-only per-doc operation log for history, periodically folded into snapshots so opening a document is always fast regardless of its age.*"

---

## 10. Consistency, latency, and caching

- **Local (your own edits): immediate, optimistic.** You never wait for the network to see your own keystroke — that's non-negotiable per the requirements. This is why the client applies its own op locally *before* the round trip.
- **Convergence across users: strong, eventually.** Every client is guaranteed to reach the *same* final document, but not necessarily at the *same instant* — there's a small window (network RTT) where two users see slightly different intermediate states. That's acceptable and unavoidable in any real-time collaborative system; what's not acceptable is *diverging* permanently, which OT/CRDT specifically prevent.
- **The sequencer's per-document order is the strong-consistency anchor.** Once the server has assigned an op a position in the canonical order, that position never changes — it's the linearization point, analogous to "payment success" in the flash-sale design.
- **Caching:** the snapshot *is* the cache — instead of a separate cache-invalidation problem, "load latest snapshot + replay recent tail" acts as a self-refreshing cache of the materialized document. There's no separate cache-vs-DB divergence to reason about because the snapshot is rebuilt directly from the authoritative op log.
- **Async, off the critical path:** persisting ops to the durable log and folding snapshots both happen after the op has already been broadcast to collaborators — nobody's typing waits on disk I/O.

---

## 11. Failure modes & graceful degradation

| Failure | What happens | Mitigation |
|---|---|---|
| Sequencer server for a doc crashes | In-flight ops for that doc's live session are lost before ack | Ops aren't considered committed until the client gets a server ack; unacked ops are resent from the client's local buffer on reconnect to a new sequencer instance |
| Snapshot job falls behind | Doc opens slower (bigger tail to replay) | Bound tail size by snapshotting more frequently as op-count grows; alert if lag exceeds threshold |
| Op log store (Kafka/Cassandra) partition unavailable | Writes for docs on that partition stall | Replication + leader failover; client-side local buffer means users keep editing locally without data loss until it recovers |
| WebSocket drops mid-session | User stops seeing others' edits, own edits queue locally | Client detects drop, buffers ops locally (same path as offline), reconnects and reconciles via OT against the version it fell behind at |
| Two users' clients both claim to be "based on" a version that no longer exists (very stale client) | Naive transform against a huge/incorrect backlog | Cap max transformable staleness; beyond it, force a fresh snapshot reload instead of transforming a huge op backlog |
| Presence pub/sub node dies | Cursors freeze/disappear briefly | No durability needed — clients simply stop seeing updates until reconnected to a healthy presence node; self-heals, no data to recover |

## 12. Follow-up curveballs

> **Interviewer:** What about rich text — bold, images, headings — not just plain characters?

**Candidate:** The same operation model extends: ops carry a target (a range or position) and an intent (`insertText`, `deleteRange`, `applyFormat(range, {bold: true})`, `insertEmbed(pos, image)`). The document is modeled as a structured tree (or a flat sequence with attribute runs), and OT's transform functions get correspondingly more complex — e.g. transforming a `delete` against an overlapping `format` requires care — but the architecture (sequencer, transform, broadcast, snapshot) doesn't change; only the operation vocabulary and transform rules grow.

> **Interviewer:** How do you support commenting/suggesting (edits that don't change the doc until "accepted")?

**Candidate:** Model suggestions as their own operation type that's stored and broadcast like any op, but the "apply to canonical rendered text" step treats it as a visual overlay rather than a structural mutation until an `acceptSuggestion` op arrives. It reuses the whole pipeline; it just adds a state ("pending") to certain ops before they're folded into the base document.

> **Interviewer:** Can you scale a single document beyond ~100 concurrent editors — say a 10,000-person live company doc?

**Candidate:** At that point the bottleneck flips from "conflict resolution" to "fan-out broadcast" — one sequencer pushing every op to 10,000 sockets. I'd introduce a fan-out tier (the sequencer publishes to a pub/sub layer, and dedicated broadcast servers hold the client connections), so the sequencer's job stays narrowly "decide order and transform," while a horizontally-scaled tier handles "deliver this op to N thousand sockets." This is the same sequencer-vs-fanout split many chat/pub-sub systems use at scale.

> **Interviewer:** How do you avoid an infinite loop of transforming against your own past ops?

**Candidate:** Every op the server sends back to the originating client is tagged so that client can recognize "this is the server's ack of my own op" and not re-apply it on top of its already-applied local copy — otherwise you'd double-insert your own character. This is exactly analogous to idempotency keys in the payment flows from other designs: the client must be able to distinguish "my op, echoed back" from "someone else's op."

---

## 13. Deep-dive: "Why not just save/sync the whole document on every keystroke?"

> **Interviewer:** Why all this operation/OT/CRDT machinery? Why not have every client just keep syncing its *whole* current document to the server on every change (or every second), and whoever's version lands last wins — last-write-wins on the document blob?

**Candidate:** This is the naive V1 from earlier, and it's worth being precise about exactly *why* it fails, because the failure isn't subtle — it happens on the very first concurrent edit, and it gets worse, not better, at scale.

### Why it fails: lost edits

```
t=0    Doc = "Hello world"                (both A and B start from this)
t=1    User A types at the end -> A's local copy: "Hello world!"
t=1    User B types at the start -> B's local copy: "Well, Hello world"
       (A and B never saw each other's edit -- both are editing from the SAME
        base state at the same moment, which is the whole point of collaboration)
t=2    A's client PUTs its whole doc: "Hello world!"      -> server stores it
t=3    B's client PUTs its whole doc: "Well, Hello world" -> server OVERWRITES A's

Final stored doc: "Well, Hello world"
A's "!" is GONE. Not merged, not conflicted-and-flagged -- just silently erased.
A sees their own screen still says "Hello world!" and has NO IDEA it was lost
until they reload and it's back to "Well, Hello world" with no exclamation mark.
```

There is no way to fix this by syncing *faster* (every keystroke instead of every second) — it makes the collision window smaller but never zero, and with real collaborators typing in different places, collisions

are the common case, not the edge case. The fundamental issue is that a whole-document snapshot **carries no information about what changed or where** — "last write wins" can only pick one blob or the other, never merge intents from both.

### Why it also fails on bandwidth

```
Whole-doc sync: every keystroke (or every tick) re-sends the ENTIRE document.
A 50KB document, 10 users, each typing ~5 keys/sec, syncing every 500ms:
  10 users x 2 syncs/sec x 50KB = ~1MB/sec of traffic for ONE document,
  almost all of it re-transmitting text that didn't change.

Operation-based sync: the same session sends ~10 users x 5 ops/sec x ~20 bytes/op
  = ~1KB/sec for the same document. ~1000x less bandwidth, and it scales with
  EDIT rate, not document SIZE -- a 500-page doc costs the same to sync as a
  1-page one.
```

### Comparison

| | | | |
|---|---|---|---|
| Concurrent edits | Silently loses one side's edit | Both preserved, transformed to compose correctly | Both preserved, merge is order-independent by construction |
| Bandwidth | O(document size) per sync, regardless of change size | O(change size) per edit | O(change size) + per-element ID metadata |
| Needs a coordinator? | No (but that's *why* it's wrong — no one decides how to merge) | Yes — central sequencer transforms in order | No — merge function is commutative/associative |
| Offline support | Terrible — reconnecting after any concurrent edit loses someone's work | Good — transform queued ops against everything missed | Great — designed for this; no central authority needed |
| Complexity | Trivial to build | Moderate-to-hard (transform functions for rich text) | Moderate (per-element IDs, tombstone GC) |
| Where it's actually fine | Single-owner documents nobody else edits concurrently (config files, personal notes with no real-time sharing) | Real-time collaborative editors with a server (Google Docs) | Offline-first / P2P / local-first apps (Figma-style, note-sync apps) |

> **Rule of thumb:** *Last-write-wins on a whole blob is only safe when you can guarantee only one writer at a time — the moment two people can edit concurrently, you need operations that carry intent (what changed, where), plus an algorithm (OT or CRDT) that can compose two intents instead of picking one and discarding the other.*

---

## 14. Deep-dive: two users insert text at the same position at the same time

> **Interviewer:** Let's go concrete. Both users are looking at the same document state. User A types `"cat"` at position 5. User B, at the exact same moment, types `"dog"` also at position 5. Neither has seen the other's edit yet. Walk me through exactly how this diverges if handled naively, and how you make it converge.

**Candidate:** This is the canonical hard case for collaborative editing — concurrent inserts at the *same* position — and it's worth walking through why naive application breaks before showing the fix.

### The setup

```
Shared base state (both clients start here), position index shown above the text:
       0    5         11
Text:  |----|"world"---|
       "Hello world"
                ^
                position 5 (right after "Hello ")

User A (locally): insert("cat ", pos=5)  -> A's local buffer: "Hello cat world"
User B (locally): insert("dog ", pos=5)  -> B's local buffer: "Hello dog world"

Both ops were generated against the SAME base version. Neither client knows about
the other's op yet. Both ops say the same thing: "insert at position 5."
```

### Why naive (order-blind) application diverges

```
If the server (or each client) just applies ops in whatever order they happen to
ARRIVE, with no transformation:

  Server receives A's op first, then B's op, applies each blindly at "position 5":
    apply A: "Hello world" -> "Hello cat world"
    apply B (blindly at pos 5, ignoring that A's insert shifted things):
      "Hello cat world" -> "Hello dog cat world"   (B's text lands BEFORE A's)

  But User A's own client, which applied its OWN op locally first and then
  receives B's op unmodified, might do:
    local: "Hello cat world"
    apply B (naively at literal pos 5 in A's local buffer):
      "Hello dog cat world"   -- looks consistent here...

  ...EXCEPT if delivery order differs elsewhere (e.g. B's client receives A's op
  before the server has decided a canonical order, or a third client receives them
  in the opposite order), different replicas apply "insert at 5" and "insert at 5"
  in different relative orders and END UP WITH DIFFERENT TEXT:
    replica 1: "Hello dog cat world"
    replica 2: "Hello cat dog world"

  Nobody's document is "wrong" per se -- both contain both edits -- but they are
  DIFFERENT from each other. That's divergence: the one thing we said we'd never
  allow. Two users looking at "the same" document now see different text, and if
  either keeps typing from their own (now-incompatible) position offsets, it
  compounds.
```

### Fix via OT: the sequencer transforms one op against the other

```
Server is the single sequencer. It receives A's op first (arbitrarily -- could be
either), so it defines the canonical order:

  1. Apply A's op as-is: insert("cat ", pos=5) on the server's canonical doc.
     Canonical doc is now "Hello cat world" at version v+1.

  2. B's op arrives, generated against the OLD version (before A's op existed).
     The server does NOT apply it blindly at pos=5. It TRANSFORMS B's op against
     A's already-applied op first:
       transform(insert("dog ", pos=5), against=insert("cat ", pos=5))
         -- tie-breaking rule (e.g. "earlier-sequenced op wins the position,
            later op's insert point shifts right by the length of the earlier
            insert"): B's effective position becomes 5 + len("cat ") = 9
       -> transformed op: insert("dog ", pos=9)

  3. Server applies the TRANSFORMED op: "Hello cat world" -> "Hello cat dog world"
     Canonical doc, version v+2: "Hello cat dog world"

  4. Server broadcasts the TRANSFORMED op (insert("dog ", pos=9), not the original
     pos=5) to every other connected client, including a correction echo to B.

  Every replica -- A's, B's, and any third viewer's -- applies the ops in the SAME
  canonical order with the SAME transformed positions, so every replica converges
  to the identical: "Hello cat dog world"
```

### Fix via CRDT: stable per-character IDs make order self-resolving

```
Instead of transforming positions, each inserted character/segment gets a unique,
totally-ordered ID at creation time, e.g. (counter, replicaId):

  A's "cat " chars get IDs: (5,A) (5,A) (5,A) (5,A)   [simplified as one block]
  B's "dog " chars get IDs: (5,B) (5,B) (5,B) (5,B)

  Every replica, whenever it sees both blocks, applies a DETERMINISTIC tie-break
  rule for equal logical positions -- e.g. "lower replicaId sorts first":
    (5,A) < (5,B)  =>  A's block is ordered before B's block, EVERYWHERE,
    regardless of which order the two inserts actually arrived over the network.

  Result, on every replica no matter the arrival order: "Hello cat dog world"
  -- same convergence as OT, but reached WITHOUT any central sequencer deciding
  "who got there first": the IDs alone make the order self-evident and identical
  on every replica that has seen both ops.
```

### Ranked approaches

1. **OT with a central sequencer (recommended for this design).** One server per document is the arbiter: it decides the canonical arrival order and performs the position transform. Simple mental model given we already have a server-backed session; battle-tested at Google Docs' scale.
2. **CRDT with stable IDs.** No coordinator needed, which is strictly more powerful when there's no always-on server (P2P, fully offline-first). Costs more per-character metadata and a trickier garbage-collection story for deleted content (tombstones), which is why I wouldn't default to it when a coordinator is available anyway.
3. **Naive order-blind apply (rejected).** As shown above, this diverges on the very first same-position concurrent insert — not a viable option, included only to show why the other two exist.

### Recommended

Go with **OT anchored on the per-document sequencer** we already need for other reasons (assigning version numbers, persisting the op log in order). It gives the same convergence guarantee as a CRDT for this deployment shape, with simpler per-character bookkeeping — the CRDT approach earns its extra complexity specifically when there's *no* server to be the sequencer, which isn't our case.

> **What to say out loud:** "*Two concurrent inserts at the same position will diverge if every replica applies them in whatever order it happens to receive them — that's the bug, not a rare edge case. OT fixes it by having one sequencer decide a canonical order and mathematically transform the later op's position to account for the earlier one, so every replica that applies ops in that same canonical order converges identically. A CRDT reaches the same convergence without a sequencer at all, by giving every inserted unit a stable, globally comparable ID so the tie-break is baked into the data instead of decided by a server. I'd pick OT here because we already have a natural per-document coordinator; I'd pick CRDT if this had to work fully offline or peer-to-peer with no server in the loop.*"

---

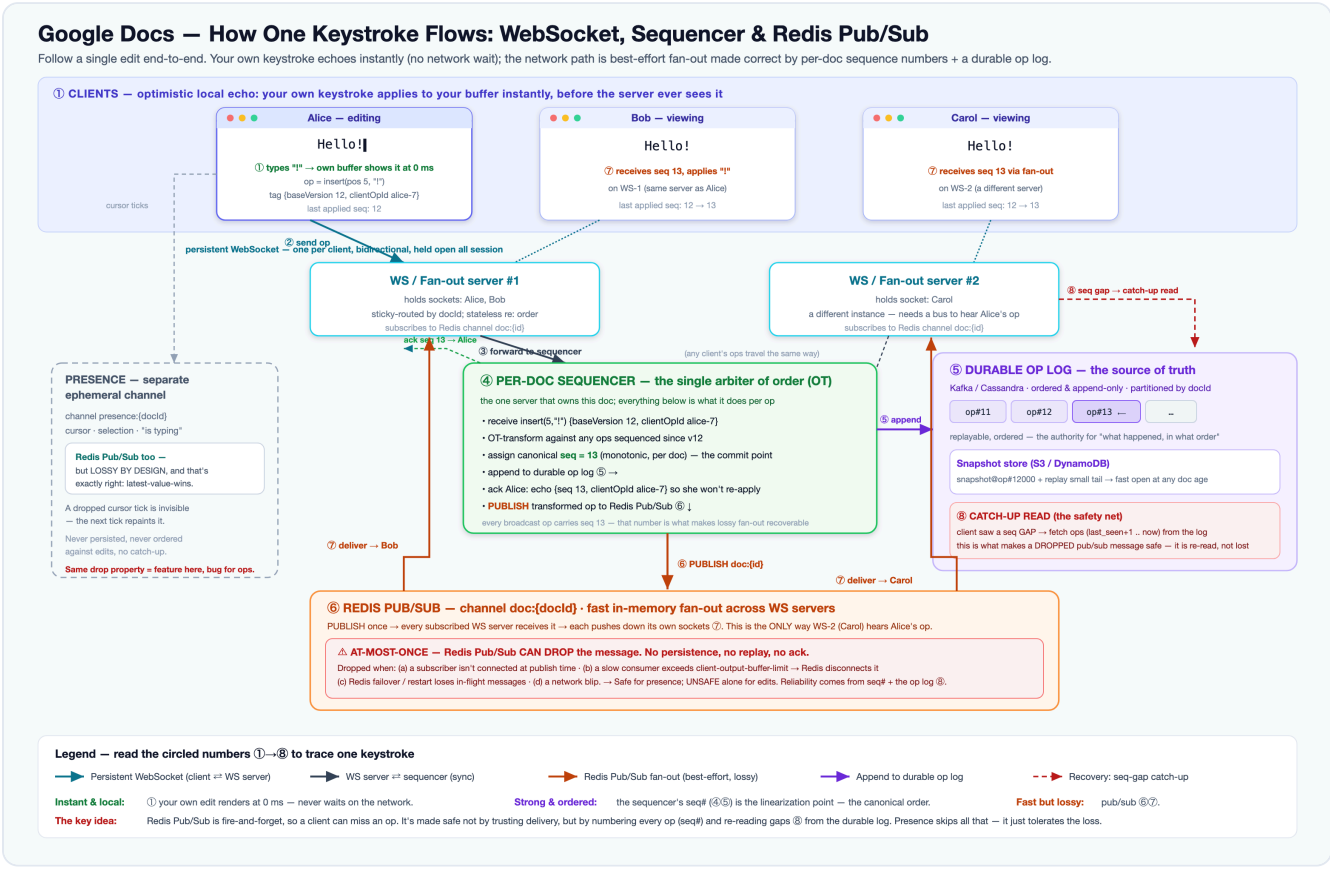
## Summary — the mental model

Google Docs is **not a throughput problem — a single document rarely exceeds a few hundred edits per second even with 100 concurrent editors. It's a convergence problem**: many writers, no lock, and a hard requirement that every replica's document ends up byte-identical despite edits racing each other over an unreliable network. The winning design: represent every edit as a small **operation**, not a document snapshot, sent over a persistent **WebSocket**; use a **central per-document sequencer running Operational Transformation** to define a canonical order and transform racing operations so both survive instead of one clobbering the other (CRDTs solve the same problem without a sequencer, which is the right trade when there's no always-on server); keep **presence** on a separate, disposable channel so it never competes with real edits; and make the document itself **event-sourced** — an append-only op log as the source of truth (which also gives you version history for free) folded periodically into **snapshots** so opening a document stays fast no matter how long its edit history is. Offline editing is not a special case: it's the same OT transform path, just reconciling a longer backlog of queued operations on reconnect.

APPENDIX — DATA FLOW · WEBSOCKET · REDIS PUB/SUB

# Google Docs — How One Keystroke Flows

Optimistic local echo · per-doc sequencer (OT) · durable op log · Redis Pub/Sub fan-out (at-most-once) made safe by sequence numbers + gap catch-up



# Google Docs — Data Flow, WebSocket & the Redis Pub/Sub Question

Companion to `HLD-Interview-Google-Docs.pdf`. The transcript nails the *algorithm* (operations, OT vs CRDT, snapshots). This companion adds what an interviewer usually probes next: **walk the data, hop by hop**, with tiny concrete examples — and it answers a specific, easy-to-get-wrong question head-on:

> **Can the WebSocket layer use Redis Pub/Sub? And can Redis Pub/Sub drop packets?**

Short answer up front, then the detail: **Yes, Redis Pub/Sub is a normal choice for fan-out between WebSocket servers — and yes, it can absolutely drop messages (it is at-most-once / fire-and-forget).** That's *fine* for presence (cursors), and *unsafe on its own* for document edits. The fix is not "make pub/sub reliable"; it's "number every op and re-read the gaps from a durable log." See `HLD-Google-Docs-DataFlow.png` / `.svg` for the picture.

---

## 1. The mental model in one breath

Three planes, deliberately kept apart:

| Plane | Carries | Guarantee | Where it lives |
|---|---|---|---|
| **Local echo** | *your own* keystroke | instant, 0 ms, no network | the browser buffer |
| **Edit sync** | everyone's ops, ordered | strong convergence (every replica ends byte-identical) | WebSocket → **sequencer** → durable **op log**; fan-out over pub/sub |
| **Presence** | cursors, selections, "is typing" | none — lossy on purpose | a separate ephemeral pub/sub channel |

The whole trick of the system is that the **fast path is allowed to be lossy** because the **slow path (the ordered op log) is the truth**, and a **sequence number** on every op lets a client notice "I missed one" and go re-read it.

---

## 2. Follow one keystroke — the end-to-end data flow

**Setup**

- Doc reads `Hello` — canonical **version 12**.
- **Alice**, **Bob**, **Carol** all have it open.
- Alice + Bob are on **WS server #1**; Carol is on **WS server #2**.
- **Alice types `!`** — trace ①→⑧ below (numbers match the diagram).

---

① **Local echo** — *0 ms, no network*

- Alice's editor inserts `!` into *her own* buffer the instant she types → she sees `Hello!`.
- Typing must **never** wait for a round trip (the hard requirement) → the client applies its own op *first*.

- The op it creates:

```
{ "type": "insert", "pos": 5, "text": "!",
  "baseVersion": 12,          // version Alice's client was at when she typed
  "clientOpId": "alice-7" }   // Alice's local id for this op (for ack matching)
```

**② Send over the WebSocket**

- Written as one WebSocket text frame onto Alice's already-open socket → WS #1.
- No new connection, no HTTP handshake — the socket has been open since she opened the doc.

**③ WS server → sequencer**

- WS #1 is only a socket-holder + fan-out relay; it does **not** decide order.
- It forwards the op to the **per-doc sequencer** — the one server that owns ordering for this doc (sticky-routed / consistent-hashed on `docId`).

**④ Sequencer — the real work** *(the linearization point)*

- **OT-transform** the op against any ops sequenced since `baseVersion 12` (here: none → unchanged).
- **Assign canonical** `seq = 13` — the moment the edit "officially happened" (like *payment success* in flash-sale). Order is now fixed forever.
- Hand off to ⑤ and ⑥.

**⑤ Append to the durable op log**

- `op#13 = insert(5,"!")` → Kafka / Cassandra, partitioned by `docId`.
- The **source of truth**; doubles as version history.
- Persisting happens **off the typing path** — collaborators don't wait on this disk write.

**Ack → Alice** *(rides back on ⑤)*

- Sequencer echoes `{seq: 13, clientOpId: "alice-7"}` down Alice's socket.
- Alice's client sees its *own* `clientOpId` → "server confirmed my already-applied op" → marks it committed, **does not re-apply** (else `Hello!!`).
- This "recognize my own echo" is exactly an idempotency key.

**⑥ Publish to the fan-out bus**

- Sequencer `PUBLISH`es the transformed op — **carrying** `seq: 13` — to Redis Pub/Sub channel `doc:{docId}`.

**⑦ Fan-out to the other clients**

- Every WS server subscribed to `doc:{docId}` receives it:
- WS #1 → pushes `seq 13` to Bob → Bob's buffer becomes `Hello!`.
- WS #2 → pushes `seq 13` to Carol → Carol's buffer becomes `Hello!`.
- This bus is the **only** way WS #2 hears an op that arrived at WS #1. No bus → Carol never sees Alice's edit.

**⑧ Gap recovery** *(the safety net)*

- Each client tracks its **last applied seq**.
- Carol at `12` suddenly receives `15` → she knows `13, 14` were **dropped**.
- She issues a **catch-up read** to the op log ("send ops `13..now`"), applies them in order → back in sync.
- **This is what makes a lossy fan-out safe.**

> Collaborators see the edit within one network RTT on the happy path (⑥⑦), and **always** converge even when the fast path drops a message (⑧). Local echo (①) never waited on any of it.

---

## 3. The durable channel — what runs at each hop, and how data flows through it

"Durable" names a **guarantee** (never lost · ordered · persisted · part of history), **not a transport**. It's a small stack of pieces, and it matters *which* piece establishes the durability.

### 3.1 The stack (and which layer actually owns durability)

| Layer | Job on the durable channel | Tech | What flows through it |
|---|---|---|---|
| **Transport** | carry ops client ⇄ server, low-latency, push | **WebSocket** | op frames tagged *must-ack* |
| **Ordering / commit** | assign canonical per-doc `seq`, run OT | **Per-doc sequencer** — stateful service, sticky-routed / consistent-hashed on `docId` (or a per-doc leader) | one op at a time → gets `seq N` |
| **Durability + history** ⭐ | the append-only, ordered, never-lost record | **Kafka** partitioned by `docId` *(or Cassandra)* | `op#N` appended in order; replayable |
| **Fast current state** | materialized doc for fast open | **S3-style object store** *(or DynamoDB / Postgres row)* keyed by `docId` | `snapshot@op#K` blobs |
| **Offline durability** | queue local ops while disconnected | **IndexedDB** (browser-local) | queued ops tagged with `baseVersion` |
| **Fan-out** *(accelerator, not durability)* | push a sequenced op to other WS servers fast | **Redis Pub/Sub** *(or Redis Streams / Kafka for replay)* | best-effort copies of `op#N` |

> ⭐ **Durability is established at the sequencer's `seq` + the Kafka/Cassandra op log — nowhere else.** WebSocket only *transports*, Redis Pub/Sub only *accelerates*, the snapshot store only *serves fast*. An op is durable the instant it's appended to the log; losing the WebSocket frame or the Pub/Sub broadcast can never un-commit it.

### 3.2 Write path — how an op *becomes* durable (and why order matters)

- **1. WebSocket frame** — the op arrives at its WS server, which forwards it to the sequencer.
- **2. Sequencer** — OT-transform, then assign `seq = N`.

- **3. Append to Kafka** (partition = `docId`) — **this is the commit point; the op is now durable + ordered.**
- **4. *Only after* the append** — ack the client, then `PUBLISH` to the Redis fan-out.

The ordering is the whole point: the **durable write happens before the broadcast**, so a dropped Pub/Sub message (or a WS server crash) can never lose a committed op — it's already in the log, and the missing client re-reads it by `seq` gap (⑧).

### 3.3 Read / open path — how a doc loads fast without replaying all history

- **1.** Load the latest **snapshot** from S3/DynamoDB (e.g. `snapshot@op#12000`).
- **2.** Replay the **small tail** `op#12001..N` from Kafka on top.
- **3.** Doc is open in bounded time — regardless of whether it has 100 or 500,000 ops of history.

Version history / point-in-time restore falls out of the same two pieces: nearest snapshot before the timestamp + replay ops up to it.

### 3.4 Offline path — durability with no network at all

- **Offline:** ops append to the browser's **IndexedDB** queue, each tagged with the `baseVersion` it was made against.
- **Reconnect:** the client resends the queued ops; the sequencer OT-transforms them against everything sequenced since `baseVersion`, appends to the Kafka log, and broadcasts — identical path to live editing, just a longer transform backlog.

---

## 4. How a WebSocket server receives data — Redis Pub/Sub (push) vs Kafka (pull)

A WS server gets ops through **two different mechanisms with different shapes**:

- **Redis Pub/Sub → live push.** The server *subscribes* and Redis pushes each op to it as it's published.
- **Kafka / Cassandra op log → pull by range.** The server *reads a bounded range of ops by* `seq` for initial load and gap catch-up. It does **not** live-subscribe to Kafka per document.

Keep that split in mind: **Redis pushes the live feed; the log is pulled on demand.**

### 4.1 What the WS server holds in memory

- `docId → { set of local client sockets }` — the fan-out routing table.
- Per socket: `lastSeqSent` — how far that client is caught up.
- One dedicated **Redis connection in SUBSCRIBE mode** (a subscribed connection can't run normal commands, so it's separate from any command connection).
- A **Kafka consumer** (or **Cassandra session**) used only for range reads — never for live per-doc delivery.

### 4.2 Live receive — from Redis Pub/Sub (push)

- When the **first** local client for `docId` connects, the server runs `SUBSCRIBE doc:{docId}`.
- Redis then **pushes** every `PUBLISH` on that channel to the server; the client library fires an on-message callback with `(channel, payload)`.
- The callback fans out to the local sockets:

```
on_message(channel = "doc:{docId}", payload = { seq, op }):
    sockets = routing_table[docId]
    for s in sockets:
        if seq == s.lastSeqSent + 1:        # contiguous → deliver
            s.send_ws_frame({ seq, op }); s.lastSeqSent = seq
        elif seq > s.lastSeqSent + 1:        # GAP → pub/sub dropped one
            trigger_catch_up(s, from = s.lastSeqSent + 1, to = seq)   # → §4.4
        # seq <= lastSeqSent → duplicate/echo, ignore
```

- When the **last** local client for `docId` disconnects, the server runs `UNSUBSCRIBE doc:{docId}` (stop paying to receive a doc nobody here is watching).

### 4.3 Initial load — when a client first opens the doc (pull)

- **1.** WS server loads the latest **snapshot** from S3 / DynamoDB ( `snapshot@op#K` ).
- **2.** WS server **range-reads the tail** `op#K+1 .. N` from the op log and folds it on top.
- **3.** Sends the client the materialized doc + `currentSeq = N` ; sets `lastSeqSent = N` .
- **4.** `SUBSCRIBE doc:{docId}` (if this server isn't already subscribed) → live updates begin.

### 4.4 Catch-up read — from Kafka / Cassandra (pull, by range)

Triggered by a `seq` gap (⑧) or a reconnect. The op log is read as a **bounded range by** `seq` , not a subscription:

- **Cassandra** (keyed by `(doc_id, seq)` ): `SELECT op FROM oplog WHERE doc_id=? AND seq >= ? AND seq <= ?` — a trivial ordered range scan. *This is exactly why keying the log by `(docId, seq)` is attractive.*
- **Kafka** (partition = `hash(docId)` ): seek the partition to the **offset** of `op#from` and read forward to `op#to` . This needs a `seq → offset` index (or a compacted/keyed topic), which makes random by-`seq` catch-up more awkward on raw Kafka than on Cassandra.
- The fetched ops are applied in order and sent down the socket; `lastSeqSent` advances to close the gap.

### 4.5 Why Redis = live-push and Kafka = pull — and the alternatives

| Bus | Mode | Good for | Durable? | Latency |
|---|---|---|---|---|
| **Redis Pub/Sub** | push ( `SUBSCRIBE` ) | the **live** fan-out hop | no (at-most-once) | sub-ms |
| **Kafka / Cassandra log** | pull (range by `seq` /offset) | initial load + **gap catch-up** | yes | ms+ |
| **Redis Streams** | push-ish ( `XREAD BLOCK` ) + pull ( `XREAD` from id) | both in one system | yes (bounded) | low |
| **Kafka consumed directly** | push (consumer subscription) | live **and** durable, no Redis | yes | higher |

- **Default:** Redis Pub/Sub for the live push (§4.2) + the log pulled for load/catch-up (§4.3–4.4). Fast where it must be, durable where it must be.

- **Consume Kafka directly for live:** each WS server is a Kafka consumer of the partitions holding its docs → durable + replayable live delivery with no Redis, but you pay consumer-group rebalancing, higher latency, and partition management. Reasonable at smaller fan-out.
- **Redis Streams:** replaces Pub/Sub as the live bus and folds catch-up in — a reconnecting server just `XREAD`s from its last-seen id, so §4.2 and §4.4 become one mechanism. Presence still stays on plain Pub/Sub.

> The through-line: **the WS server is a router.** It receives ops by *pushed subscription* (Redis, fast/lossy) and by *pulled range read* (the log, durable/exact), reconciles them with each socket's `lastSeqSent`, and writes WebSocket frames down to the browsers. The `seq` number is the glue that lets the two receive paths cooperate without duplicating or losing an op.

---

## 5. Why a WebSocket (and not HTTP polling)?

Editing is **bidirectional and continuous**: the server must *push* you other people's ops the instant they happen, and you must push yours the instant you type. A persistent WebSocket gives:

- **Server push** — someone else's op arrives without you asking. HTTP can't do this without long-polling hacks.
- **No per-message HTTP overhead** — a keystroke op is ~20 bytes; wrapping each in a fresh HTTP request is absurd.
- **Low, predictable latency** — polling every 200 ms both *feels* laggy and wastes requests when nobody's typing.

One socket per client per doc session, held open the whole time, carrying three multiplexed message kinds: **ops** (durable, ordered), **acks**, and **presence** (ephemeral).

---

## 6. The main event: can WebSocket use Redis Pub/Sub, and can it drop packets?

### 6.1 Why a bus is needed at all

At any real scale you run **many WebSocket server instances** behind a load balancer, and the clients of one document are **spread across several of them** (Alice+Bob on WS #1, Carol on WS #2). An op that lands on WS #1 must reach the sockets held by WS #2. So you need a **message bus between the WS servers**. Redis Pub/Sub is a classic, low-latency choice: each WS server `SUBSCRIBE`s to `doc:{docId}`; the sequencer (or the receiving WS server) `PUBLISH`es; Redis fans it out to all subscribers in-memory, sub-millisecond.

### 6.2 Yes — Redis Pub/Sub can drop your message

Redis Pub/Sub is **at-most-once, fire-and-forget**. There is *no* persistence, *no* per-subscriber queue, *no* ack, *no* replay. A published message is delivered only to subscribers **connected at that instant**, and even then delivery isn't guaranteed. Concretely, a message is lost when:

| # | Scenario | What happens |
|---|---|---|
| a | A WS server **isn't subscribed at publish time** (just restarted, mid-reconnect, network partition) | it never receives that op — Redis does not hold it for later |
| b | A subscriber is **slow** and its output buffer fills | Redis enforces `client-output-buffer-limit pubsub`; when exceeded, Redis **force-disconnects** the subscriber, dropping everything queued |
| c | **Redis fails over / restarts** | in-flight messages are gone; a replica promoted on failover never had them (pub/sub isn't replicated like keyspace data) |
| d | A transient **network blip** between publisher/Redis/subscriber | the message evaporates silently — you won't even get an error |

So: **you must assume the fan-out layer loses messages.** This is not a Redis misconfiguration to fix; it's the defined semantics of Pub/Sub.

### 6.3 Why that's totally fine for presence

Presence (cursor position, selection, "is typing") is **latest-value-wins and disposable**. If Carol misses three of Alice's cursor moves, nobody notices — the fourth update repaints Alice's cursor in the right place. Presence is *never persisted, never ordered against edits, never caught up.* Here, the drop property is a **feature**: no durability machinery, no memory growth, self-healing by construction. Redis Pub/Sub (or the WS server's own in-memory fan-out) is the *right* tool precisely because it's cheap and lossy.

### 6.4 Why it's unsafe *alone* for document edits — and how to make it safe

For edits, a dropped op is a disaster: that client is now **permanently missing a change** and will **diverge** — the one thing OT/CRDT exist to prevent. But the fix is **not** to make pub/sub reliable. The fix is to stop *trusting delivery* and instead make loss *detectable and recoverable*:

1. **Number every op.** The sequencer stamps a **monotonic per-doc** `seq` on every broadcast op (④).
2. **Clients track "last applied seq."** Ops are contiguous: after `12` you expect `13`.
3. **Detect the gap.** If a client receives `seq 15` while sitting at `12`, it *knows* `13,14` were dropped — the sequence number turned an invisible loss into a visible, precise gap.
4. **Recover from the durable log.** The client issues a **catch-up read** — "send me ops `13..now`" — from the op log (or asks its WS server to backfill), applies them in order, and converges (⑧).
5. **Client→server direction is acked + resent.** Alice's op isn't considered committed until she gets the `{seq}` ack; unacked ops are re-sent on reconnect (this is the same path as offline editing).

Net effect: **pub/sub provides *speed* (best-effort, instant), the seq# + op log provide *correctness* (at-least-once, ordered, convergent).** The lossy fast path is a cache/accelerator in front of the authoritative log — never the system of record.

### 6.5 If you'd rather the *bus* handle recovery: Redis Streams (or Kafka)

You can push the gap-recovery down into the transport instead of a separate op-log fetch:

| Option | Delivery | Replay / catch-up | Ordering | Cost / notes |
|---|---|---|---|---|
| **Redis Pub/Sub** | at-most-once (**drops**) | none | per-publish only | cheapest, lowest latency; perfect for presence, needs seq#+log for ops |

| Option | Delivery | Replay / catch-up | Ordering | Cost / notes |
|---|---|---|---|---|
| **Redis Streams** | at-least-once | **yes** — `XREAD` / `XAUTOCLAIM` from last-seen id; consumer groups | ordered per stream | a reconnecting WS server reads *from where it left off*; must `XADD MAXLEN` /trim to cap memory |
| **Kafka** | at-least-once | yes — offset replay, long retention | ordered per partition | heavier + higher latency than Redis; usually already your durable op log, so you may fan out straight from it |

A very common production shape: **Redis Pub/Sub for the low-latency edit fan-out *and* the presence channel, with the Kafka/Cassandra op log + seq# as the correctness backstop.** If you dislike the separate catch-up path, swap the edit fan-out to **Redis Streams** so "resume from last id" is built in. Presence stays on plain Pub/Sub either way — it neither needs nor wants replay.

---

## 7. Worked example — a dropped op, recovered

```
Carol is at seq 12.  Alice makes two quick edits → sequencer assigns seq 13, seq 14.

  seq 13  PUBLISH doc:{id}  ──► WS#2 delivers to Carol   ✅ Carol now at 13
  seq 14  PUBLISH doc:{id}  ──✗  (Carol's WS#2 was mid-reconnect — Redis dropped it)
  seq 15  PUBLISH doc:{id}  ──► WS#2 delivers to Carol   ⚠ Carol receives 15 while at 13

Carol's client: "expected 14, got 15 → GAP."
  → catch-up read to op log: GET ops(14..15)
  → apply op#14, then op#15 in order
  → Carol now at seq 15, converged.  No edit lost, no divergence.
```

Notice what did the work: **the sequence number** (made the loss *visible*) and **the durable log** (made it *recoverable*). Redis Pub/Sub did its job — fast in the happy case — and its failure was caught, not catastrophic.

---

## 8. Presence data flow (the deliberately-lossy twin)

```
Alice moves her cursor to offset 8
   │  (throttled to ~10/s; no local "echo" needed — it's her own cursor)
   ▼
WebSocket frame {type: "presence", cursor: 8, sel: [8,8]}  → WS #1
   ▼
PUBLISH presence:{docId}  (Redis Pub/Sub — SEPARATE channel from doc:{docId})
   ▼
every WS server delivers latest-value to its sockets → Bob & Carol repaint Alice's caret
   ✗ if any tick is dropped: ignored — the next tick (≤100ms later) repaints correctly
```

Same Redis Pub/Sub, opposite philosophy: **no seq#, no log, no catch-up.** Keeping presence on its own channel is what protects edit latency — a storm of cursor ticks never competes with the ordered, must-not-lose edit pipeline.

## 9. Where each guarantee is anchored (quick reference)

| Concern | Mechanism | Consistency |
|---|---|---|
| Your own keystroke shows instantly | optimistic local apply (①) | immediate, local |
| Everyone converges to identical bytes | sequencer's canonical order + OT transform (④) | strong, eventual |
| An op is never lost | durable op log + client ack/resend + seq-gap catch-up (⑤⑧) | at-least-once → exactly-once effect |
| Others' edits arrive fast | Redis Pub/Sub fan-out (⑥⑦) | best-effort (accelerator) |
| Cursors/selections | separate ephemeral pub/sub | none (lossy by design) |
| Fast open of an old doc | snapshot + replay recent tail | — |

**The single sentence to say out loud:** "*Redis Pub/Sub is a fine fan-out for the WebSocket tier, but it's at-most-once — it can drop an op — so I never rely on it for correctness. Every op carries a per-doc sequence number and is appended to a durable ordered log; clients detect a seq gap and re-read the missing ops from that log. Pub/sub buys latency, the log buys correctness. Presence rides the same kind of pub/sub channel but deliberately tolerates loss, because a stale cursor is repainted by the next update.*"